

AEC 저-SMI 완화 게이트 전체 도출 파이프라인

AEC 저-SMI 완화(De-escalation) 게이트 — 전체 도출 파이프라인(`main_aec_full_derivation_pipeline_simplified.py`) 알고리즘 정리

대상 파일: `code/past/main_aec_full_derivation_pipeline_simplified.py` (2,005 lines)

이 파일은 다른 파이프라인(`1_aec_residual_reclassify.py` 등)처럼 "모델을 학습하는" 스크립트가 아니라, 최종 논문(manuscript)에 실린 해석 가능한 4-region AEC 게이트 규칙이 정확히 어떤 탐색 과정을 거쳐 도출됐는지를 처음부터 끝까지 재현하는 "도출 기록(derivation record)" 스크립트다. 즉 "왜 R1~R4 구간을 저 위치로 잘랐는지", "왜 하필 이 4개의 feature/부호/폭/가중치 조합인지", "왜 16가지 +/- 패턴 중 이 5개만 양성으로 인정하는지"에 대한 답을, 코드 상단에 상수로만 박아두지 않고 실제 탐색 알고리즘(Section 6)을 다시 돌려서 눈으로 확인할 수 있게 만든 파일이다.

1. 큰 그림: 이 파일이 하는 일

```
python main_aec_full_derivation_pipeline_simplified.py --mode reproduce      # 최종 확정 규칙만 빠르게 재현
python main_aec_full_derivation_pipeline_simplified.py --mode full-search    # 탐색 과정(01~04 CSV) 까지 전부 재생성
python main_aec_full_derivation_pipeline_simplified.py --mode plot-1x3      # internal/external 1x3 평균곡선 그림만
```

`main()` 은 `--mode` 와 무관하게 마지막에 항상 `run_plot_1x3_mean_curves()` 와 `MDCARD_main()` 을 호출한다. 즉 실행할 때마다:

- (reproduce) 확정된 규칙만 다시 계산해 최종 지표 CSV/그림/JSON 생성, 또는 (full-search) region scout → branch screen → combo 패턴 탐색 → 확정 규칙까지 전 과정 재생성
- internal/external 코호트별 1x3 평균곡선 + R4 접선(tangent) + 2x3 mirror-deviation 그림 생성
- `outputs/MD` 에 사람이 읽기 좋은 "요약 카드" PNG 3장 생성 (원본 스크린샷과 재현치 대조 포함)

을 항상 수행한다.

1.1 라벨 정의 (다른 파이프라인과 동일)

```
SMI = TAMA / (Height[m])^2
Low-SMI(y=1): 남성 SMI < 45.4, 여성 SMI < 34.4
```

`load_dataset()` (line 226)에서 계산되며, 이 파일 전체에서 그대로 "정답(ground truth)"으로 쓰인다.

1.2 이 파일이 다루는 "게이트"의 개념

1. **1차 임상 스크리닝** — 나이/키/몸무게/성별만으로 로지스틱 회귀를 만들어, 민감도(sensitivity)를 일정 수준($S80/S85/S90 = 80/85/90\%$) 이상으로 강제한 임계값에서 "Clinical+"를 정의한다.
2. **2차 AEC 형태(morphology) 게이트** — Clinical+ 로 판정된 사람들 중에서, AEC-128 곡선의 특정 구간(R1~R4)이 특정 방향의 형태를 보이면 "이 사람은 사실 저위험(AEC+)일 가능성이 크다"고 보고 최종 판정을 **양성 → 음성으로 완화(de-escalate)** 한다.
3. Clinical- 로 판정된 사람은 건드리지 않는다. AEC 게이트는 오직 Clinical+ 그룹 내부의 위양성(FP)을 줄이는 2차 필터로만 쓰인다 (`evaluate_deescalation` 의 `post_positive = clinical_positive & ~aec_positive`).

2. 상수 섹션(Section 0, line 60-195) — 변수 하나하나가 어떻게 정해졌는가

이 섹션의 상수는 성격이 서로 다른 **세 종류**로 나뉜다. 이걸 구분해서 봐야 "어떻게 값이 정해졌는지"가 명확해진다.

(A) 연구자가 직접 고른 "설계/탐색 하이퍼파라미터" — 데이터로부터 계산된 값이 아니라, 탐색 공간을 정의하기 위해 미리 정한 값

변수	값	의미 / 어떻게 정해졌나
SEED	20260629	전역 RNG(<code>RNG</code>), CV fold shuffle, section 8 전용 RNG(<code>CV_RNG</code>)의 시드. 특정 통계적 근거는 없고 재현성을 위해 고정된 임의의 정수 (날짜 형식처럼 보이지만 임의값). 이 값이 바뀌면 5-fold 분할이 바뀌어 임상 점수·임계값·게이트 결과가 미세하게 달라질 수 있음 → 반드시 고정.
SMOOTHING_SIGMA	1.0	AEC-128 원본 곡선에 적용하는 가우시안 스무딩 (<code>ndimage.gaussian_filter1d</code>)의 표준편차. slice 단위 노이즈를 줄이기 위한 전처리 강도이며, 다른 파이프라인 (<code>1_aec_residual_reclassify.py</code>)과 동일한 값을 그대로 재사용.
TARGET_OPS	[("S80",0.80), ("S85",0.85), ("S90",0.90)]	임상 모델의 목표 민감도(operating point) 3종. "최소 이 정도 민감도는 반드시 확보한다"는 임상적 요구사항(저-SMI를 놓치면 안 되므로 민감도 최우선)에서 나온 표준적인 스크리닝 임계값 후보들.
PRIMARY_OP	"S90"	세 operating point 중 논문의 주 분석(primary analysis)으로 채택된 지점 . S90(민감도 90% 확보) 이 최종 게이트가 적용되는 기준.
NI_MARGIN	0.05	민감도 손실에 대한 공식 비열등성(non-inferiority) 허용 한계 = 5%p . AEC 게이트가 아무리 특이도를 올려도, "놓치는 저-SMI 환자 비율의 97.5%/95% 단측 신뢰구간 상한"이 이 값을 넘으면 그 규칙은 기각된다 (<code>formal_NI_pass</code> , <code>pass_both</code> 의 핵심 기준). 임상적으로 통용되는 5%p 비열등성 마진 관례를 채택.

변수	값	의미 / 어떻게 정해졌나
DESCRIPTORS	12종 (level_mean , level_sd , endpoint_delta , linear_slope , slope_mean , slope_sd , abs_slope_mean , abs_slope_max , curv_mean , curv_sd , abs_curv_mean , abs_curv_max)	각 구간(region)에서 뽑을 수 있는 후보 요약 통계량 목록. 레벨(평균/표준편차), 구간 시작-끝 변화량, 선형 추세, 1차 미분(기울기)·2차 미분(곡률)의 평균/표준편차/절대값 평균·최댓값까지 — "이 구간의 모양을 숫자로 어떻게 요약할 수 있는가"에 대한 탐색 후보 전체 목록.
SIGNS	[-1, 1]	각 feature가 "AEC+ 방향"으로 작동할 때 어느 부호로 봐야 하는지의 탐색 격자 (증가가 저위험 신호인지, 감소가 저위험 신호인지 모르므로 둘 다 탐색).
WIDTHS	[0.35, 0.50, 0.70]	branch_gate_score 의 가우시안 국소화 폭(아래 4.3절) 탐색 격자. 임상 점수가 임계값 근처에서 얼마나 넓은 범위까지 AEC 보정을 받을지를 결정.
LAMDAS	[0.25, 0.40, 0.55, 0.70]	branch_gate_score 에서 AEC feature가 임상 점수를 얼마나 세게 밀어낼 수 있는지의 가중치 탐색 격자 (locked-region 브랜치 스크리닝 단계에서 사용, 4개 값으로 촘촘히).
REGION_SCOUT_LAMDAS	[0.25, 0.55, 0.70]	위와 같은 개념이지만 region scout(임의 구간 탐색) 단계 에서 쓰는 더 coarse한(3개) 격자 — 탐색해야 할 구간(window) 후보가 매우 많으므로(coarse+fine grid) 계산량을 줄이기 위해 lambda 격자를 줄임.
REGION_SCOUT_SCORE_WEIGHTS	(1.0, 0.30, 0.20, 0.40)	region scout 단계에서 후보 (구간, 부호, 폭, lambda) 조합의 순위를 매기는 점수식의 가중치 (w_mean_acc, w_min_acc, w_min_spec, w_sens_loss). score = w_mean_acc*평균 정확도 개선 + w_min_acc*최악 operating point 정확도 개선 + w_min_spec*최악 특이도 개선 - w_sens_loss*최악 민감도 손실. 탐색 단계이므로 민감도 손실 페널티(0.40)를 상대적으로 크게 뒤서 "민감도를 심하게 깎는 구간"을 초반에 걸러낸다.
BRANCH_SCREEN_SCORE_WEIGHTS	(1.0, 0.35, 0.20, 0.25)	4개 locked region이 이미 정해진 뒤, 그 안에서 (descriptor, 부호, 폭, lambda) 브랜치 후보 순위를 매기는 점수식 가중치. region scout보다 w_min_acc (0.35)의 비중을 높이고 w_sens_loss (0.25)의 비중은 낮췄다 — 이 단계는 뒤이은 combo 탐색(Section 6, run_combo_pattern_search)에서 민감도 손실을 NI 마진으로 다시 엄격하게 걸러내므로, 브랜치 선별 단계 자체는 정확도 개선 쪽에 조금 더 무게를 둔다.

이 표의 값들은 전부 "탐색 공간을 얼마나 넓게/좁게 잡을지"를 정하는 **연구자 설정값**이며, 특정 데이터 계산으로 유도된 값이 아니다. 값을 바꾸면 탐색 결과(어떤 구간/브랜치/패턴이 뽑히는지)가 달라질 수 있지만, 알고리즘 자체의 정합성에는 영향이 없다.

(B) 데이터 기반 탐색으로 "도출"된 후 고정(locked)된 상수 — 이 파일의 핵심

이 상수들이 바로 "논문에 실린 최종 규칙"이며, Section 6의 탐색 알고리즘을 실제로 돌리면 왜 이 값들이 선택됐는지 재현할 수 있다 (자세한 도출 절차는 4절 참고).

변수	값	어떻게 정해졌나
LOCKED_REGIONS	R1=45-56, R2=57-80, R3=97-128, R4=117-128 (128칸 중 slice index; "Cranio-caudal index: 1 inferior pubic margin -> 128 liver dome")	run_region_scout() 가 폭넓은 구간 후보(coarse: 길이 16/24/32 slice를 8칸 간격, fine: 길이 12~32 slice를 33번 slice 이후 4칸 간격)를 전수 스캔해 01_region_scout_window_feature_ranked.csv 로 순위를 매긴 뒤, "광범위 구간 스카우트 결과 + 시각적 해석"을 거쳐 사람이 최종적으로 4개 구간으로 확정한 것(주석: "chosen after broader region scout searches and visual interpretation"). 즉 완전 자동 선택이 아니라 데이터 기반 후보 목록 + 도메인 판단의 결합.
LOCKED_PRIMARY_BRANCHES	R1: endpoint_delta, sign -1, width 0.50, λ 0.25 / R2: level_mean, sign -1, width 0.70, λ 0.25 / R3: linear_slope, sign +1, width 0.35, λ 0.25 / R4: endpoint_delta, sign -1, width 0.50, λ 0.25	screen_branch_candidates() 가 4개 locked region × 12개 descriptor × 2개 부호 × 3개 width × 4개 λ (=총 288조합/구간)를 모두 평가해 구간별 상위 6개를 추리고 (02_locked_region_branch_screen.csv, 03_selected_branch_candidates_for_combo_search.csv), 이어서 run_combo_pattern_search() 가 R1~R4 후보 조합 × "16개 4-bit 코드 중 정확히 5개를 +로 인정하는 패턴 부분집합"(C(16,5)=4,368 가지)을 전수 평가해, internal/external 양쪽 모두에서 공식 비열등성 (NI_MARGIN=0.05) 통과 + 특이도 개선(+) + 정확도 개선(+) 을 만족하는 조합만 남긴다(04_combo_pattern_search_s90_top200.csv). 이 파일의 LOCKED_PRIMARY_BRANCHES 는 바로 그 통과 조합 중 논문이 최종 채택한 하나이며, 04_locked_primary_rule_row_from_search.csv 는 탐색 결과표에서 정확히 이 조합에 해당하는 행만 뽑아 보여준다.
LOCKED_PRIMARY_PATTERNS	{"++++", "+---", "+--+ ", "-+-+", "----+"} (5개)	위와 같은 combo 탐색에서, pattern_masks_exactly_k(5) 로 생성한 "16가지 4-branch +/- 조합 중 정확히 5개를 AEC+로 인정" 패턴 부분집합 전수(4,368가지) 중, 위 4개 브랜치 조합과 짝지어 NI/특이도/정확도 기준을 통과한 패턴 집합. 정렬 기준은 formal_NI_5pp_both_pass (통과 여부) → external_acc_gain → external_spec_gain → internal_acc_gain 내림차순이며, 이 순서상 상위인 조합이 최종 채택됨.
REGION_SPANS	LOCKED_REGIONS 와 동일 구간 + 그림용 색상/투명도 (#4E79A7 등)	순수 시각화용 상수. 구간 경계는 LOCKED_REGIONS 와 동일해야 하므로 별도 리스트로 중복 정의되어 있음(그림 색상 정보 추가).

(C) 외부에서 계산되어 "그대로 고정 이식"된 참조값 — 이 파일 안에서 재계산 불가

변수	값	어떻게 정해졌나
CNN_PROBABILITY_NPZ / CNN_S90_INDEX / CNN_BRANCH_THRESHOLDS / CNN_SELECTED_PATTERNS	CNN_S90_INDEX=2 , CNN_BRANCH_THRESHOLDS= [0.80,0.60,0.90,0.60] , CNN_SELECTED_PATTERNS= { "+---", "---+", "-+- +", "++++" }	2차(secondary) "CNN-mimic" 게이트용 상수. 원래 CNN을 직접 학습하던 별도 스크립트가 단일 파일 병합 과정에서 삭제되었기 때문에, 이 파일은 CNN을 다시 학습하지 않고 사전 계산된 확률 파일 (surrogate_mimic_balanced_probabilities.npz)만 읽어서 과거 스크린샷(outputs/MD/144838527.png)의 결과 (S90 de-escalated n=40 internal / ~51-52 external, TP lost=2/1)를 재현하도록 임계값·패턴을 고정해둔 것. 주석에 "surrogate_mimic_summary.json의 winners로 바꾸지 말 것"이라고 명시 — 그건 더 최근의 다른 brute-force 재탐색 결과라 규칙 자체가 다르기 때문.
MDCARD_MD_ORIGINAL_PRIMARY , MDCARD_MD_ORIGINAL_CNN	협업자가 공유한 원본 스크린샷의 수치(문자열)	코드로 재계산할 수 없는 "예전에 실제로 보고됐던 값"이라서 상수로 박제해두고, 이 파일이 재계산한 값과 나란히 비교해 "재현 성공 여부"를 표로 보여줌 (reproduction_check_vs_MD_original.png).

3. 데이터 로딩 및 전처리 (Section 1, line 198-247)

1. `aec_columns()`: `aec_1 ~ aec_128` 컬럼만 골라 숫자 순서로 정렬.
2. `matrix_from_aec_sheet()`: 결측/비유한값(NaN, inf 등)을 **해당 컬럼(slice)의 중앙값**으로, 컬럼 중앙값마저 없으면 **전체 행렬의 전역 중앙값**으로 대체.
3. `load_dataset()`:
 - `raw`: 위에서 만든 128-slice 원본 행렬
 - `smooth`: `SMOOTHING_SIGMA=1.0` 가우시안 스무딩 적용 (`mode="nearest"` 로 경계 보정)
 - `norm = patient_wise_mean_normalize(smooth)`: 환자별 128-slice 평균으로 나눠 **환자 간 절대 스케일 차이를 제거**하고 곡선의 "형태(shape)"만 남김 (평균이 0/비유한이면 나눗셈 방지를 위해 1.0으로 대체)
 - `sex`, `smi`, `low_smi`: 1.1절의 라벨 정의 그대로 계산

4. 임상 모델 (Section 2, line 250-384)

1. `clinical_design_matrix()`: `PatientAge`, `Height`, `Weight`, `sex_M(남=1)` 4개 변수를 사용. 결측은 **internal(내부) 코호트의 중앙값**으로 채우고, ****internal의 평균/표준편차로 표준화(z-score)****한 뒤 동일한 평균/표준편차를 `external(외부) 코호트`에도 그대로 적용(외부 데이터는 표준화 파라미터 산출에 전혀 관여하지 않음 — 데이터 누수 방지).
2. `stratified_folds()`: 클래스(저-SMI 여부)별로 셔플 후 5-fold에 균등 배분하는 수동 stratified split (RNG는 `SEED` 로 고정된 것을 사용).

3. `fit_clinical_scores()`: `LogisticRegression(C=1e6, ...)` — **C=1e6**은 사실상 정규화가 거의 없는(**un-regularized**) 로지스틱 회귀를 의미(변수가 4개뿐이라 과적합 위험이 낮다고 보고 정규화를 최소화). 5-fold로 OOF(out-of-fold) 점수를 만들고, 내부 전체로 재학습한 최종 모델을 외부에 적용.
4. `z_standardize_by_internal()`: 임상 점수(decision function 값)를 **internal 점수의 평균/표준편차로 다시 z-표준화** — 이후 모든 게이트 로직(`branch_gate_score` 등)이 이 z-점수 스케일 위에서 동작한다.
5. `threshold_for_min_sensitivity()`: 각 목표 민감도(S80/S85/S90)에 대해 "**민감도 $\geq$ target을 만족하는 임계값 후보 중 특이도가 가장 높은 값**"을 선택 (없으면 저-SMI 환자군 점수의 `1-target` 분위수로 대체).
6. `make_context()`: 위 전부를 묶어서 internal/external 데이터, 라벨, 임상 z-점수, 3개 operating point 별 임계값(`thresholds`), 그리고 그 임계값 기준 Clinical+ 마스크(`cpos_g`, `cpos_s`)까지 한 번에 만든다.
 - **주의**: `make_context()` 는 프로세스 한 번 실행에 **두 번 호출**된다(메인 파이프라인 1회 + `MDCARD_main()` 내부 1회). 전역 RNG가 공유되면 두 번째 호출 때 시드가 이미 소모돼 fold 분할이 달라지므로, 매번 `np.random.default_rng(SEED)` 로 **새로 재시딩**해서 호출 순서와 무관하게 항상 동일한 결과가 나오도록 설계돼 있다(line 361-368 주석 참고).

5. AEC 형태(feature) 추출 (Section 3, line 387-473)

1. `d1()` / `d2()`: 128-slice 곡선의 1차 미분(기울기, slope)과 2차 미분(곡률, curvature). 맨 앞 값은 그 다음 diff 값을 복사해 길이를 원본과 맞춤.
2. `_region_descriptors()`: 주어진 구간(block)에 대해 12종 descriptor를 계산 (2.A절 `DESCRIPTORS` 표 참고).
 - `linear_slope` 는 구간 내 slice 인덱스를 중심화한 뒤 OLS 기울기 공식($\Sigma(x \cdot y) / \Sigma x^2$)으로 직접 계산.
 - `include_min_max=True` 일 때만(=region scout 전용) `level_min/level_max` 도 추가 — locked region descriptor 행렬에는 포함되지 않음.
3. `window_features()` vs `locked_region_descriptor_matrix()`: 전자는 **임의의 (start,end) 구간 목록**에 대해(=region scout용), 후자는 **고정된 R1~R4**에 대해 같은 descriptor 계산 로직을 재사용.
4. `standardize_features_by_internal()`: feature 행렬도 임상 변수와 동일한 패턴 — internal의 중앙값/평균/ 표준편차로 표준화 파라미터를 만들고 external에 그대로 적용.
5. `candidate_windows()`: region scout이 스캔할 구간 후보 생성기. `run_region_scout()` 에서
 - coarse: `step=8`, 길이 `[16,24,32]`, 전체(1~128) 범위
 - fine: `step=4`, 길이 `[12,16,20,24,28,32]`, 33 번 slice 이후만 두 그리드를 합쳐 사용 (뒤쪽 구간을 더 촘촘히 보는 이유는 R3/R4처럼 후반부 구간이 최종적으로 중요했기 때문으로 보이며, 전반부는 성긴 그리드로 대략적인 스크리닝만 수행).

6. 게이트 스코어링 수식 (Section 4, line 476-580) — 알고리즘의 핵심

```
def branch_gate_score(clinical_z, feature_z, threshold, sign, width, lam):
    boundary = exp(-0.5 * ((clinical_z - threshold) / width) ** 2) # 임계값 근처에서만 1에 가까운 가우시안 종모양
    return clinical_z + lam * boundary * (sign * feature_z)

def branch_vote(...):
    return branch_gate_score(...) < threshold # 보정된 점수가 임계값보다 낮아지면 "+"(=AEC 이상 형태) 투표
```

- **직관:** 임상 점수(`clinical_z`)가 임계값에서 멀리 떨어진 사람(확실히 양성/확실히 음성)은 `boundary` ≈ 0 이라 AEC 정보가 전혀 영향을 못 준다. **딱 경계선 부근에 있는 애매한 환자들에게만** AEC feature(`feature_z`, 방향은 `sign`)가 점수를 밀어줄 수 있다. `width` 는 "경계선 부근"의 폭을, `lam` 은 그 밀어주는 힘의 세기를 결정한다.
- 4개 브랜치(R1~R4) 각각 독립적으로 `vote(+/-)`를 계산 → 4비트 문자열 패턴("++++" 등, `vote_pattern_from_matrix`)으로 합침 → `LOCKED_PRIMARY_PATTERNS` (5개)에 속하면 `aec_positive` = True.
- `evaluate_deescalation()`: `post_positive = clinical_positive & ~aec_positive` 로 최종 판정을 만들고,
 - **민감도 손실** = `tp_lost / total_low` (원래 Clinical+였던 저-SMI 환자 중 AEC+로 완화되어 놓친 비율)
 - `clopper_pearson_one_sided_upper()` 로 **95% 단측 신뢰구간 상한**을 구해 `NI_MARGIN(0.05)` 과 비교 → `formal_NI_pass`
 - **특이도 개선** = `fp_removed / total_nonlow` (Clinical+였지만 저-SMI가 아니었던 사람 중 올바르게 완화된 비율)
- `conditional_low_smi_table()`: Clinical+ 안에서 "AEC+(완화군)"과 "AEC-(유지군)"의 실제 저-SMI 발생률을 Fisher's exact test로 비교 — AEC+ 군의 발생률이 유의하게 낮아야 게이트가 임상적으로 타당하다는 근거.

7. 확정 규칙 도출(Section 6, line 683-1020) — 4단계 탐색 알고리즘

이 섹션이 바로 2절 (B)의 상수들이 어떻게 나왔는지를 실제로 재현하는 코드다. `--mode full-search` 로 실행하면 다음 4단계를 순서대로 수행한다.

Stage 1 — `run_region_scout()`: 어느 구간(window)이 유망한가

- 5절의 coarse+fine 구간 후보(수백 개) × 12 descriptor × `SIGNS` (2) × `WIDTHS` (3) × `REGION_Scout_LAMBDAS` (3) 조합 각각에 대해, 3개 operating point(S80/S85/S90)에서 단일 브랜치만 적용했을 때의 성능 (`summarize_single_feature_rule`: 평균/최소 정확도 개선, 최소 특이도 개선, 최대 민감도 손실)을 계산.
- `REGION_Scout_SCORE_WEIGHTS` 로 점수를 매겨 내림차순 정렬 → `01_region_scout_window_feature_ranked.csv`.
- **이 결과 자체가 최종 게이트는 아니다** — "어떤 구간대가 후보로 유망한지"에 대한 데이터 근거를 제공하며, 여기에 시각적 해석을 더해 `LOCKED_REGIONS` (R1~R4)가 확정됐다.

Stage 2 — `screen_branch_candidates()`: R1~R4 안에서 어느 `descriptor`·부호·폭·λ가 유망한가

- 이번엔 구간을 `LOCKED_REGIONS` 4개로 고정하고, $12 \text{ descriptor} \times \text{SIGNS} (2) \times \text{WIDTHS} (3) \times \text{LAMBDA} (4) =$ 구간당 288조합을 `BRANCH_SCREEN_SCORE_WEIGHTS` 로 채점 → `02_locked_region_branch_screen.csv`.
- 구간별 점수 상위 6개만 골라 `selected` 에 담고 (`03_selected_branch_candidates_for_combo_search.csv`), `LOCKED_PRIMARY_BRANCHES` 에 있는 4개는 상위 6개 안에 없더라도 강제로 포함시켜(line 808-827), 다음 단계에서 "확정 규칙이 후보 공간 어디에 있는지"를 반드시 확인할 수 있게 한다.

Stage 3 — `precompute_branch_votes()`: 후보별 투표 결과 사전 계산

- 선택된 각 브랜치 후보에 대해 `internal/external` × 3개 operating point 각각에서 `branch_vote()` 를 미리 계산해둔다(반복되는 combo 탐색에서 매번 다시 계산하지 않기 위한 캐싱).

Stage 4 — `run_combo_pattern_search()`: 브랜치 조합 × 패턴 조합 전수 탐색

- R1 후보 × R2 후보 × R3 후보 × R4 후보의 모든 조합(`itertools.product`)에 대해:
 - 4개 브랜치의 +/- 투표를 4-bit 코드(0~15)로 압축(`code_from_four_votes`)
 - `pattern_masks_exactly_k(5)`: 16개 코드 중 정확히 5개를 +로 인정하는 부분집합 전체 ($C(16,5)=4,368$ 가지)에 대해, 벡터화된 방식(`evaluate_all_masks_from_counts`)으로 민감도 손실/특이도 개선/정확도 개선을 한 번에 계산
 - 채택 조건(`pass_both`): `internal-external` 양쪽 모두 `upper95 ≤ NI_MARGIN(0.05)` 그리고 특이도 개선 > 0 그리고 정확도 개선 > 0
 - 통과한 (브랜치 조합, 패턴 조합) 쌍만 결과에 남기고, `formal_NI_5pp_both_pass` → `external_acc_gain` → `external_spec_gain` → `internal_acc_gain` 순으로 정렬해 상위 200개를 `04_combo_pattern_search_s90_top200.csv` 로 저장.
 - 정확히 `LOCKED_PRIMARY_BRANCHES` + `LOCKED_PRIMARY_PATTERNS` 에 해당하는 행만 추려 `04_locked_primary_rule_row_from_search.csv` 로 별도 저장 — "확정 규칙이 실제로 이 탐색에서 통과했다"는 것을 직접 보여주는 감사(audit) 자료.

이 4단계를 거치고 나면, 2절 (B)의 `LOCKED_REGIONS` / `LOCKED_PRIMARY_BRANCHES` / `LOCKED_PRIMARY_PATTERNS` 가 "임의로 정해진 상수"가 아니라 "이 탐색 알고리즘을 실제로 돌려서 나온, 공식 기준을 통과한 결과 중 하나를 논문이 채택한 것"임이 재현된다.

8. 확정 규칙 적용 및 출력물 (Section 5, line 583-679)

- `compute_locked_gate()`: `LOCKED_PRIMARY_BRANCHES` 4개를 `internal/external` 각각에 적용해 `aec_positive` 마스크 생성.

- `compute_secondary_cnn_mimic()`: 2절 (C)의 CNN 참조 상수로 2차 게이트 재현 (npz 파일 없으면 `None`).
- `write_final_outputs()`:
 - `final_s90_primary_and_cnn_metrics.csv` — primary/secondary × internal/external 4행의 전체 지표
 - `final_clinical_positive_aec_split.csv` — Clinical+/AEC+ vs Clinical+/AEC- 그룹별 저-SMI 발생률 + Fisher p
 - `final_external_s90_1x3_r4_tangent.png` — 7절 그림
 - `final_summary.json` — 확정 규칙 자체(브랜치/패턴/임계값)를 JSON으로 재저장

9. 그림 (Section 7, line 1023-1132)

`make_main_figure()` 가 external 코호트 기준 1x3 패널을 그린다:

- **A. Clinical S90 operating point** — Clinical+ vs Clinical- 평균 곡선
- **B. Outcome phenotype** — 실제 저-SMI(+) vs 비저-SMI 평균 곡선
- **C. Conditional AEC split with R4 fitted tangent** — Clinical+ 안에서 AEC+ vs AEC- 평균 곡선, R4 구간(117-128)에서 각 그룹 평균 곡선에 1차 직선을 적합(`np.polyfit`)해 기울기를 텍스트로 표시 (`add_r4_tangent`) — R4가 `linear_slope / endpoint_delta` 기반 브랜치의 근거임을 시각적으로 보여줌.

모든 패널 배경에는 `add_region_spans()` 로 R1~R4 구간이 색칠되어 표시된다. x축은 "Craniocaudal index: 1 inferior pubic margin -> 128 liver dome" 인덱스(머리-발 방향).

10. Section 8 — internal/external 1x3 평균곡선 (별도의 독립 백엔드)

이 섹션(line 1135-1551)은 원래 별개 파일(`main_plot*_s90_core_1x3_mean_curves.py`, 지금은 삭제됨)을 병합한 것으로, 이 파일 자체의 `make_context() / fit_clinical_scores()` 와는 다른, 독립된 임상 스코어링 계산(`LSG_load_dataset`, `LSG_clinical_scores`, 접두어 `LSG_`)을 사용한다.

- `LSG_OPS` 에는 `S82.5`, `S87.5` 가 추가로 들어있어 원본 스크립트와 동일한 5개 operating point를 계산.
- 별도 RNG(`CV_RNG = np.random.default_rng(CV_SEED)`, `CV_SEED` 도 동일하게 `20260629`)를 써서, 이 섹션의 fold 분할이 (1) 원본 독립 스크립트가 프로세스 시작 시 단 한 번만 계산했던 것과 동일하게 나오고, (2) 이 파일의 메인 파이프라인이 이미 전역 `RNG` 를 얼마나 소비했는지와 무관하게 항상 동일하게 나오도록 보장한다.
- `LOCKED_PRIMARY_BRANCHES / LOCKED_PRIMARY_PATTERNS / REGION_SPANS` 처럼 두 백엔드 사이에서 **진짜로 동일한 순수 상수·수학 함수**(`branch_gate_score`, `mean_ci`, `plot_group_mean`, `style_axis` 등)만 공유하고, 임상 점수 자체는 절대 섞이지 않는다(주석: "두 백엔드는 서로 다른 모델/feature bank 라 `clinical_z`가 호환되지 않는다").

- `render_cohort()` 가 코호트별로 1x3 PNG, R4 tangent 포함 1x3 PNG, 2x3 mean+mirror-deviation PNG, 요약 CSV 2개, JSON 1개를 생성한다. **Mirror-deviation plot**(`plot_mirror_deviation`)은 기준 곡선 대비 두 그룹의 절대편차를 위(빨강)/아래(파랑)로 대칭 배치해, "두 그룹이 기준선에서 얼마나, 어느 구간에서 벌어지는지"를 한눈에 비교하기 위한 그림이다.

11. MD 요약 카드 (line 1601-2000)

`MDCARD_main()` 은 `outputs/*/MD/` 에 사람이 읽기 좋은 요약 카드 PNG 3장을 만든다:

1. `aec_1x3_primary_gate_summary.png` — 코호트 기초 통계 + 1차(interpretable 4-region) 게이트 지표
2. `cnn_mimic_secondary_gate_summary.png` — 2차 CNN-mimic 게이트 지표 (npz 없으면 생략)
3. `reproduction_check_vs_MD_original.png` — 2절 (C)의 `MDCARD_MD_ORIGINAL_PRIMARY / _CNN` 하드 코딩 값과, 이 스크립트를 재실행해서 얻은 값을 나란히 놓고 "일치/근사일치"를 색상(초록/황색)으로 판정

`MDCARD_draw_card()` 는 matplotlib 텍스트만으로 표 형태 카드를 그리는 순수 렌더링 헬퍼이며(첫 컬럼 너비를 실제 텍스트 폭 측정으로 자동 조절), 계산 로직과는 무관하다.

12. 데이터 누수 방지 설계 요약

- 임상 변수 표준화 파라미터, feature 표준화 파라미터, 임계값(S80/S85/S90), 브랜치/패턴 선택 — **전부 internal 코호트에서만 결정**되고 external에는 고정 적용만 됨.
- external 코호트는 Section 6의 탐색(구간 스카우트, 브랜치 스크리닝, combo 탐색)에서도 **평가 지표 계산에는 등장하지만, 그 지표가 임계값/구간/브랜치 자체를 선택하는 기준으로 쓰이지는 않는다** — 다만 `pass_both` 조건 자체가 "internal *그리고* external 양쪽 모두 통과"를 요구하므로, external 성능도 최종 규칙 채택에 실질적인 제약으로 작동한다(완전한 held-out은 아니고, "양쪽에서 모두 일반화되는 규칙만 채택"하는 이중 검증 구조).
- Section 8의 별도 백엔드는 자체 RNG(`CV_RNG`)를 써서 메인 파이프라인의 전역 RNG 소비 순서와 무관하게 항상 동일한 fold 분할을 재현하도록 설계됨(9절 참고).